

G07 GATE A - ParcelPulse Fulfillment

NORMAL FINAL ATTEMPT

GATE A ASSESSMENT CONTRACT

- Normal route starts with Gate A only.
- Save a genuine attempt before protected Review.
- Repair the smallest weak competency.
- Gate B/C are fresh reassessments only when explicitly assigned.
- All K1-K8 + zero unresolved critical failures; no percentage shortcut.

FINAL COURSE-CONTENT PRACTICAL - BEGINNER-FIRST NAVIGATION, CLOSED-BOOK PERFORMANCE

Static package can prove controls. Only a real learner attempt + human/Mentor review can prove Gate pass.

This book is your teacher

Purpose

This guide explains the assessment sequence and evidence contract without exposing protected totals, accepted IDs, incident diagnosis or solution code.

Start state

M19 complete and Gate A explicitly active.

Closed-book boundary

During the first attempt do not open protected Review, protected controls, reference answers, AI solving, or another learner solution. The package starter is intentionally incomplete.

What success looks like

A small local data system whose requirements, lineage, latest-state logic, quality decisions, outputs, rerun behavior, incident recovery, controlled change, production translation and defense are all independently evidenced.

Contents / Index

Assessment contract

Page 2

Contents / Index

Page 3

Scenario + public requirements

Page 4

Source + first-use map

Page 5

Phase 1 - baseline build

Page 6

Phase 2 - validation + idempotence

Page 7

Phase 3 - incident + controlled change

Page 8

Phase 4 - production + architecture + job defense

Page 9

Submission + protected route

Page 10

Scenario + public requirements

Domain

ParcelPulse Fulfillment | entity: shipment | party: customer

Service level

daily by 08:00; source may redeliver files

Sensitive field

customer email is PII. Serving output must not expose contact email/PII.

Baseline serving statuses

SHIPPED, DELIVERED

Required output files

`serving\_current.csv`, `quarantine.csv`, `run\_manifest.json`, `validation\_report.json` - create these in your learner workspace, not in the source directory.

Runtime truth

Local gate harness proves local execution only. Spark/Kafka/Fabric/cloud claims require separate observed evidence from the relevant modules/projects.

Source + first-use map

source_events.csv

Raw event stream/snapshot input. Preserve checksum and row-level lineage.

parties.csv

Party reference containing PII. Use only what is necessary; do not publish email.

settlements.jsonl

Settlement/claim/invoice-like records used in reconciliation.

source.db

SQLite copy for SQL reasoning/validation; access via Python sqlite3 or another SQLite client.

sql_workspace.sql

Five unsolved SQL prompts. Fill your own SQL answers separately or in a copied workspace file.

pipeline_skeleton.py

Unsolved Python starter. COPY/CREATE your implementation in learner workspace; do not treat starter as answer.

public_contract_check.py

Public structural checker. It tests output shape/PII/duplicates/value validity but intentionally hides expected totals.

Phase 1 - baseline build

1. Inventory + lineage

Record every source path, row grain, key fields and checksum before transformation. State what one source row represents.

2. Deterministic latest state

Define the latest-state grain and deterministic ordering/tie-break. Prove one current row per business entity without hiding multiplication with DISTINCT.

3. Explicit acceptance/quarantine

Translate public business rules into visible accepted/quarantine reasons. Never silently drop a bad record.

4. PII-safe publish

Create required serving and quarantine outputs. Serving must exclude contact email/PII. Keep deterministic column/order rules where practical.

5. SQL + Python evidence

Save code, SQL, commands, logs and exact output paths. The Gate is not complete because a script ran once.

Phase 2 - validation + idempotence

Source controls

Record source event count, distinct entity count and lineage/checksum controls. Do not use hidden expected values.

Latest-state control

Independently verify one latest row per entity and that stale records cannot win.

Accept/quarantine reconciliation

Prove accepted + quarantined latest entities reconcile to the latest-state population; explain every quarantine reason.

Settlement/value reconciliation

Independently compare computed business value to the relevant successful settlement/claim/invoice logic. A mismatch must be visible and blocking where required.

Idempotence

Rerun identical raw inputs from a clean/repeatable state and prove the logical outputs/control values are unchanged. Record both runs.

Public structural checker

Run ``python 01_GATE_A/G07_GATE_A_SOURCE/public_contract_check.py <your_serving_current.csv>``. Exit 0 is only a structural check - it does NOT prove hidden expected counts/totals or Gate pass.

Phase 3 - incident + controlled change

Do not open early

The supplied incident and change request files are staged assessment events. Open each only when the Gate sequence instructs you and after the prior evidence checkpoint is saved.

Incident sequence

Save baseline -> open incident -> append supplied incident events -> observe symptom from evidence -> form hypothesis -> choose smallest safe recovery -> rerun validation -> preserve pre/post evidence.

Changed requirement sequence

Only after recovery evidence is saved: open change request -> state expected effect BEFORE implementation -> change configuration/logic -> rerun clean baseline inputs -> prove only intended behavior changed -> preserve old history/evidence.

Critical rule

A later success never erases the original failure. Keep attempt and incident history append-only.

Phase 4 - production + architecture + job defense

K4 orchestration/quality

Explain dependencies, retries/backfill boundary, blocking checks, tests and delivery/handoff.

K5 storage/distributed/lakehouse/streaming

Explain file/partition choices, Spark/Delta translation, event-time/offset/replay concerns and when batch vs stream is justified.

K6 cloud/security/observability/cost

Explain managed cloud/Fabric translation, PII/secrets, least privilege, metrics/logs/alerts and cost controls without claiming unobserved runtime.

K8 architecture

State requirements -> decisions -> at least two rejected alternatives -> failure domains -> limitations -> handoff runbook.

Job-ready defense

Point to one strongest M18 project, one M19 interview/technical-English evidence item, and answer follow-up questions from evidence rather than memorized slogans.

Submission + protected route

Evidence file

FILL a copied `evidence\_template.json` as `08\_WORKSPACE/A/attempt-XX/evidence.json`. Every claim must point to real code/output/log/diagram/recording evidence.

Structural checker

`python 07\_EVIDENCE\_RECORD/evaluate\_evidence.py 08\_WORKSPACE/A/attempt-XX/evidence.json` -> exit 0 means structurally complete for human review only.

Exact route

If all K1-K8 are technically demonstrated and no critical failure remains, PASS ends G07. If weak, save A first, then Review A may open; B remains sealed until explicit fresh-reassessment assignment.

Never use a percentage shortcut

One unresolved critical failure blocks pass even if many other dimensions are strong. Human/Mentor review decides technical correctness.